

Podstawy UML

W artykule opisuję podstawy UML. Po lekturze tego artykułu poznasz kilka rodzajów diagramów, które moim zdaniem są najbardziej przydatne. Dowiesz się w jakich sytuacjach UML może być dobrym narzędziem. W artykule stawiam raczej na praktyczne zastosowanie niż rygorystyczną zgodność ze specyfikacją UML.

<https://www.samouczekprogramisty.pl/podstawy-uml/>

Spis treści

- [Czym jest UML](#)
- [Czy warto uczyć się UML'a](#)
- [Czy UML jest używany w pracy związanej z oprogramowaniem](#)
- [Narzędzia](#)
- [Najczęściej używane diagramy UML](#)
 - [Diagram klas](#)
 - [Klasa](#)
 - [Relacje](#)
 - [Relacje dwukierunkowe](#)
 - [Diagram komponentów](#)
 - [Czym jest komponent](#)
 - [Przykładowy diagram komponentów](#)
 - [Diagram wdrożenia](#)
 - [Elementy diagramu wdrożenia](#)
 - [Przykładowy diagram wdrożenia](#)
 - [Diagram sekwencji](#)
 - [Elementy diagramu sekwencji](#)
 - [Rodzaje komunikatów](#)
- [Dodatkowe materiały do nauki](#)
- [Podsumowanie](#)

Jak mówi znane powiedzenie „jeden obraz jest wart tysiąca słów”. Takie przypadki zdarzają się także w programowaniu. Często w trakcie projektowania czy rozmawiania na temat fragmentu oprogramowania programistom dużo łatwiej jest się porozumieć rysując. Takie rysunki mogą opisywać ogólną architekturę projektu, sposób podejścia do rozwiązania, kolejność zdarzeń w ramach procesu itd. Dobrze jest mieć wspólny język. W tym przypadku pomocny może być UML.

Czym jest UML

UML to akronim pochodzący od angielskiego określenia *Unified Modeling Language*. W polskim tłumaczeniu znany jest jako zunifikowany język modelowania. UML to jasno wyspecyfikowany język składający się z kilkunastu rodzajów diagramów. Diagramy te pozwalają na formalne opisywanie i modelowanie struktur czy procesów.

Czy warto uczyć się UML'a

Odpowiadając na tak postawione pytanie w jednym zdaniu mogę powiedzieć, że z mojego doświadczenia UML jest ważny i warto znać jego podstawy. Chociażby po to żeby rozszerzyć swój „słownik”, który później możemy użyć w trakcie rozmowy na temat programowania z inną osobą. UML to kolejne narzędzie, które możesz używać w odpowiednich sytuacjach. Rysunek, który usuwa zbędne szczegóły pokazując najbardziej istotne aspekty jest niezastąpiony.

Jednak to tylko część rzeczywistości. UML jest ważny, między innymi z wyżej wspomnianych powodów. Jednak ten sam UML to „kobyła”. [Specyfikacja UML w wersji 2.5.1](#) zawiera 754 strony! Pracując jako programista od 2007 roku w całej swojej karierze nie spotkałem ani jednej osoby, która fanatycznie przestrzegałaby reguł opisujących UML'a¹. Część funkcjonalności UML'a bardzo rzadko albo w ogóle nie jest wykorzystywana w praktyce.

Innymi słowy: tak, warto poznać UML'a, jednak wybiórczo.

Czy UML jest używany w pracy związanej z oprogramowaniem

Na początku muszę powiedzieć Ci trochę o moich doświadczeniach. Po kilku latach pracy zauważyłem, że nie czuję się dobrze w korporacjach. Projekty, które wykorzystują „ciężkie metodologie” do ich prowadzenia też raczej nie są dla mnie. Mimo pracy jako programista od 2007 roku doświadczyłem wyłącznie niedużej części dużego świata firm IT. Bardzo możliwe, że w środowisku, którego nie lubię nacisk na „czystego UML'a” jest większy.

Z mojego doświadczenia UML jest wykorzystywany w nieformalny sposób. To tak jak z językiem obcym – najważniejsza jest komunikacja. Możesz robić mnóstwo błędów, jeśli jednak potrafisz się dogadać z drugą stroną to jesteś w domu. Właśnie komunikacja i umiejętność przekazywania informacji jest tu kluczowa. Innymi słowy jeśli będziesz znać podstawy najbardziej istotnych diagramów, to ta wiedza powinna być wystarczająca.

Taki punkt widzenia potwierdza też badanie przeprowadzone na grupie programistów, testerów, architektów czy kierowników projektów. Wynik przeprowadzonej ankiety potwierdza, że UML używany jest raczej nieformalnie.

Narzędzia

UML to diagram, czyli rysunek. Do efektywnej pracy przyda Ci się zestaw narzędzi pozwalający tworzyć te diagramy.

Przede wszystkim polecam tablicę i marker (lub kartkę i długopis). To zdecydowanie najczęściej używane narzędzia przy pracy z nieformalnymi diagramami.

W trakcie pracy używam programów [UMLet](#) i [yED](#). Są to darmowe programy, które pozwalają na tworzenie niektórych rodzajów diagramów UML. Istotne w nich dla mnie jest to, że same programy są proste a tworzone diagramy zapisane są w postaci tekstowej (można je eksportować do formatów graficznych). Format tekstowy świetnie nadaje się do zapisania w repozytorium git'a.

Istnieje całkiem sporo narzędzi, które mają dużo większe możliwości, jednak dla komercyjnych zastosowań są płatne.

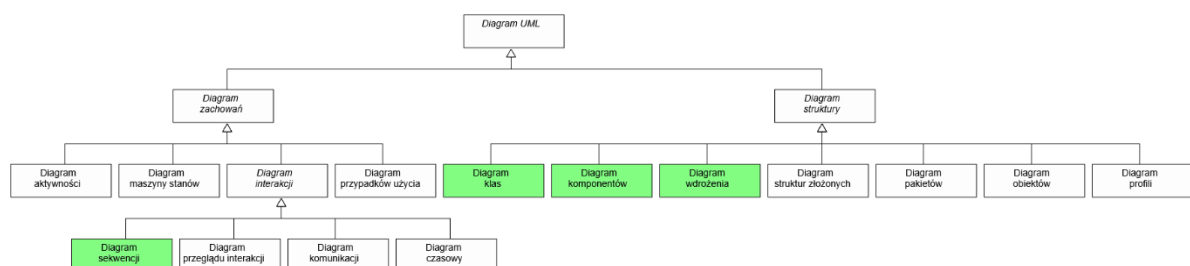
Najczęściej używane diagramy UML

Wspominałem to już wcześniej, jednak powtórzę to po raz kolejny. Poniżej prezentuję wyłącznie podzbiór diagramów. Skupiam się wyłącznie na tych, które doczekały się swojego praktycznego zastosowania w mojej dotychczasowej pracy komercyjnej.

Pomijam diagramy, które wymagane były tylko w trakcie projektów na uczelni.

Dodatkowo nie opisuję tu wszystkich możliwości, a jedynie te najczęściej używane.

Poniżej pokazałem diagram UML, który zawiera wszystkie dostępne diagramy. Na zielono oznaczyłem wyłącznie te, które opisuję w tym artykule:



Diagramy w UML

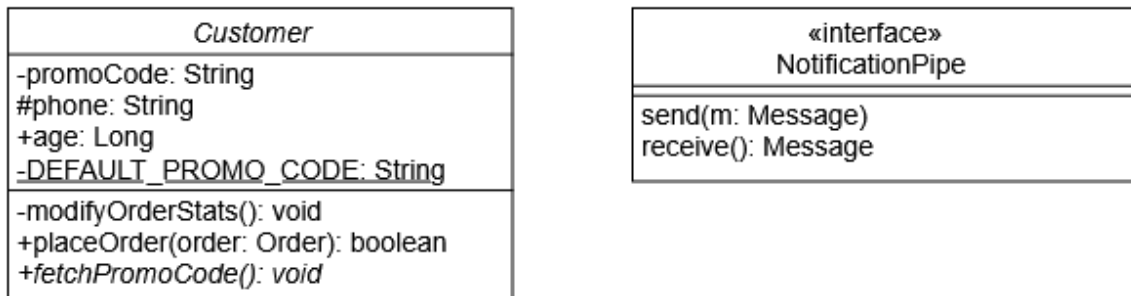
Diagram klas

Diagram klas (ang. *class diagram*) to chyba najczęściej używany diagram. Służy do pokazania klas i zależności między nimi. Pozwala na szczegółowy opis klas zwracając uwagę na dostępne atrybuty i operacje. Ta szczegółowość pozwala na generowanie kodu

na podstawie kompletnego diagramu. W praktyce nigdy nie spotkałem się z takim zastosowaniem. Diagram klas pozwala na „narysowanie” wycinka większego systemu. Jest on jednym z najbardziej rozbudowanych diagramów w notacji UML.

Klasa

Zacznę od pokazania symbolu klasy:



Klasa w diagramie klas

Klasa reprezentowana jest przez prostokąt podzielony na kilka części. W pierwszej z nich znajduje się nazwa klasy. W przykładzie jest to Customer. Następna sekcja zawiera atrybuty, kolejna metody.

Elementy, które są podkreślone oznaczają elementy statyczne. Na przykład atrybut DEFAULT_PROMO_CODE jest statycznym atrybutem klasy. Elementy pisane kursywą są abstrakcyjne (może dotyczyć także samej klasy), na przykład metoda fetchPromoCode jest abstrakcyjna.

Zarówno atrybuty jak i operacje mogą być poprzedzone symbolem. Dopuszczalne są między innymi:

- + element publiczny,
- # element „chroniony” (może odpowiadać protected w języku Java),
- – element prywatny.

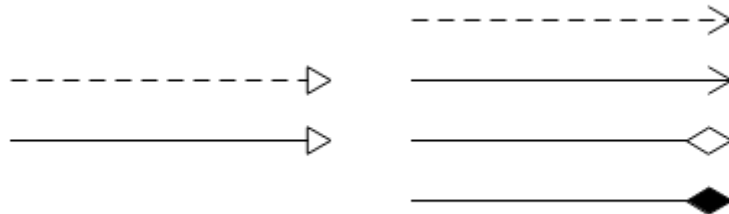
Klasa w przykładzie ma cztery atrybuty. Trzy atrybuty instancji i jeden atrybut klasy (statyczny). Atrybuty zapisywane są w formacie *nazwa:typ*. Ta sama klasa ma trzy metody: prywatną metodę `modifyOrderStats()` i dwie metody publiczne. Zwróć uwagę na to, że metody mogą mieć określone typy parametrów i wartości zwracane.

W podobny sposób oznacza się interfejs. W odróżnieniu od klasy zawiera on tak zwany stereotyp «interface». Na diagramie powyżej `NotificationPipe` jest interfejsem zawierającym dwie metody. Zauważ, że w tym przypadku pominąłem symbole określające dostępność metod.

Atrybuty klas mogą być także opisane przez relacje pomiędzy klasami.

Relacje

Pomiędzy klasami mogą występować relacje. Przykładem relacji jest [dziedziczenie](#). Relacje reprezentowane są przez różne symbole. Proszę spojrz na rysunek poniżej, na którym zebrałem możliwe relacje:



Przykładowe pomiędzy klasami

Zacznę od lewej kolumny. Pierwsza przerywana strzałka reprezentuje *implementację*. Jest używana do tego żeby pokazać jaki interfejs jest implementowany przez klasę. Druga oznacza *dziedziczenie*. W tym przypadku grot wskazuje klasę nadrzędną.

W prawej kolumnie znajdują się strzałki pokazujące relacje pomiędzy klasami inne niż implementacja czy dziedziczenie. Posegregowałem je w rosnąco według tego jak silne są relacje przez nie opisywane.

Relacje ze strzałkami mogą być jednokierunkowe albo dwukierunkowe. W przypadku relacji jednokierunkowej strona bez grotu używa strony, na którą pokazuje grot. W przypadku braku grotu relacja jest dwukierunkowa. Trochę inaczej sprawa wygląda z rombami. Opiszę to jak poznasz rodzaje relacji.

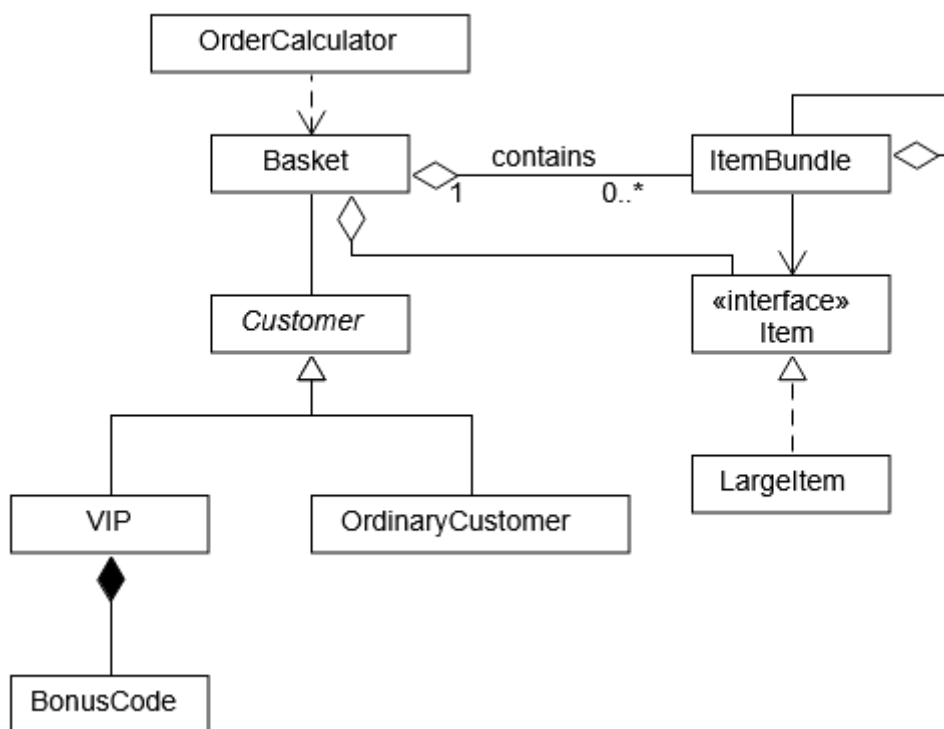
Najstańszą relacją pomiędzy klasami jest *zależność*. Reprezentowana jest przez przerywaną linię. *Zależność* oznacza, że jedna klasa w pewnym momencie używa innej, na przykład jako parametr, czy wartość zwracana metody. W przypadku *zależności* klasa, od której zależy nie jest zapisana jako atrybut. Przykładem *zależności* w bibliotece standardowej Javy może być zależność `Integer` od `String`, widzieć ją na przykład w metodzie `Integer.valueOf(String)`.

Kolejnym rodzajem relacji jest *asocjacja*. W tym przypadku jest to zapis, który może zastąpić atrybut klasy – jeśli nie chcesz dodawać atrybutu w prostokącie reprezentującym klasę możesz użyć *asocjacji*. Przykładem *asocjacji* w bibliotece standardowej Javy może być `FileInputStream` i `String`. Klasa `FileInputStream` posiada atrybut typu `String` reprezentujący ścieżkę do pliku.

Kolejną relacją jest *agregacja*. *Agregacja* wprowadza w relacji stronę, która jest „właścicielem”. Jedna klasa agreguje inną. Relacja tego typu oznaczona jest przez ciągłą linię z pustym rombem po stronie właściciela. W bibliotece standardowej tego typu relacja występuje pomiędzy `ArrayList` a klasą, której instancje przechowuje.

Ostatnią relacją jest *kompozycja*. *Kompozycja* jest bardzo podobna do *agregacji*. Jest między nimi jedna znacząca różnica. W przypadku *kompozycji* „właściciel” jest odpowiedzialny za tworzenie (cykl życia) elementów, które grupuje. Przykładem *kompozycji* w bibliotece standardowej Javy może być implementacja `HashMap`, która zarządza elementami w kolekcji opakowując je w instancje `HashMap.Node`, które tworzy.

Proszę spójrz na diagram poniżej (dla czytelności pominąłem w nim atrybuty i operacje). Pokażę Ci na nim przykładowe relacje pomiędzy klasami:



Możliwe relacje pomiędzy klasami

- klasa `LargeItem` implementuje interfejs `Item` – *implementacja*,
- klasy `VIP` i `OrdinaryCustomer` dziedziczą po klasie abstrakcyjnej `Customer` – *dziedziczenie*,
- klasa `OrderCalculator` używa klasy `Basket` – *zależność*,
- klasa `Basket` wie o kliencie z którym jest powiązana (klasie `Customer`), odwrotne stwierdzenie także jest prawdziwe – *asocjacja*,
- klasa `Basket` może zawierać wiele instancji klasy `Item` – *agregacja*,
- klasa `VIP` zawiera wiele instancji klasy `BonusCode` i zarządza ich cyklem życia – *kompozycja*.

Wiesz już, że strzałeczka oznacza kierunek relacji. Na przykład asocjacja pomiędzy `ItemBundle` a `Item` jest jednokierunkowa. `ItemBundle` wie o powiązanej klasie `Item`, `Item` zaś nie wie nic o `ItemBundle`. Jeśli strzałeczka nie jest umieszczona

oznacza to, że relacja jest dwukierunkowa – można „przejsć” z jednej klasy do drugiej w obu kierunkach.

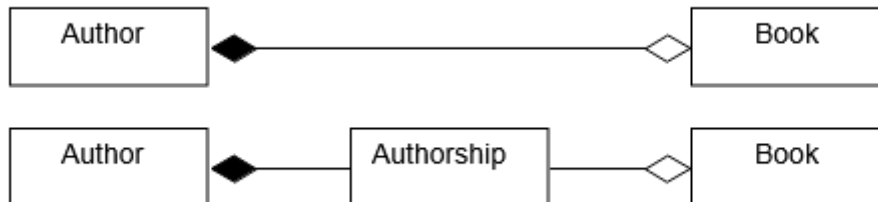
Trochę inaczej wygląda sprawa w relacjach *agregacji* i *kompozycji*. W tym przypadku romby oznaczają stronę, która agreguje drugą stronę relacji. Na powyższym przykładzie klasa `VIP` zarządza cyklem życia `BonusCode`. `BonusCode` nic nie wie o klasie `VIP`.

Nowością dla Ciebie jest także komentarz do relacji (`contains`), który może ją opisywać. Nowe są także oznaczenia pokazujące licznosc. W powyższym przykładzie jeden koszyk może zawierać wiele elementów (0..*).

Relacje dwukierunkowe

Wiesz już jak oznaczana jest dwukierunkowa relacja *zależności*. Na przykładzie wyżej pokazałem ją pomiędzy klasami `Customer` i `Basket`. W praktyce występują też bardziej zagmatwane przypadki. Wyobraź sobie klasę reprezentującą książkę – `Book`. Książka ma autora – `Author`. Jedna książka może być napisana przez wielu autorów, a jeden autor może napisać wiele książek. To klasyczna relacja „wiele do wielu”.

Często tego typu relacje wprowadzają nową klasę, która reprezentuje samą relację. W tym przypadku byłoby to autorstwo – `Authorship`. Poniższy diagram pokazuje przykładowe sposoby przedstawienia sytuacji tego typu na diagramie UML.



Przykład relacji dwukierunkowych

W pierwszym przypadku `Author` przechowuje kolekcję `Book` i zarządza ich cyklem życia. `Book` wie o liście swoich autorów.

W drugim przypadku `Author` przechowuje kolekcję swoich „autorstw”. Podobną kolekcję przechowuje także `Book`.

Diagram komponentów

Wiesz już, że diagram klas pozwala zobaczyć powiązania pomiędzy klasami w wąskiej części systemu. Diagram komponentów (ang. *component diagram*) pozwala spojrzeć na projekt z większej odległości. W diagram komponentów kluczową rolę odgrywają komponenty. Proszę spójrz na przykładowy symbol komponentu:



Przykładowy komponent

Jak widzisz komponent to prostokąt ze specyficzną ikonką w prawym górnym rogu. Komponent na rysunku wymaga jeden interfejs i sam dostarcza dwa. Komponent `UserManagement` wymaga dostępu do interfejsu `persistence` a sam zapewnia dwa inne `register` i `ban`.

Interfejs to kreska z kółkiem (interfejs udostępniany przez komponent) lub kreska z półkołem (interfejs wymagany przez komponent). Relacje pomiędzy komponentami odbywają się poprzez interfejsy. Można powiedzieć, że komponenty łączy relacja *zależności* – najstabsza z typów relacji występująca w diagramie klas.

Czym jest komponent

Wiesz już jak wygląda symbol komponentu i interfejsów. Tylko czym ten komponent właściwie jest? Cytując za specyfikacją:

A Component represents a modular part of a system that encapsulates its contents and whose manifestation is replaceable within its environment.

Powyższe zdanie można przetłumaczyć jako: komponent reprezentuje wydzieloną, opakowaną część systemu, której reprezentacja jest wymienna w ramach swojego środowiska.

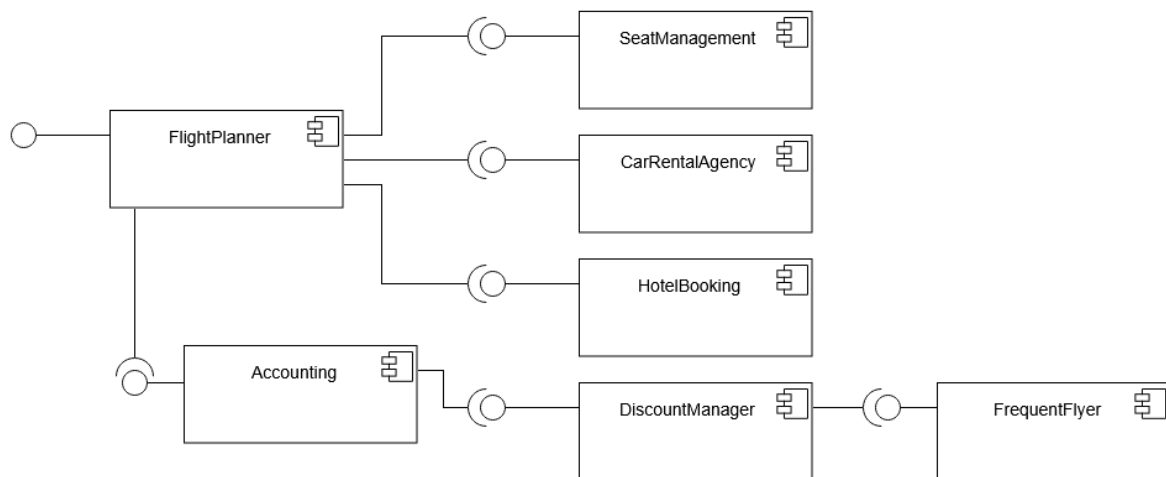
A teraz raz jeszcze, moimi słowami. Komponent to część systemu, która ma swoje interfejsy. Interfejsy czyli dokładnie określone sposoby komunikacji. Interfejsy służą do komunikacji z pozostałymi komponentami. Każdy z komponentów można zastąpić inną implementacją. Istotne jest to, że każda implementacja musi spełniać wymagania dotyczące jego interfejsów.

Jak widzisz definicja komponentów jest dość luźna. Do tego worka można wsadzić bardzo dużo rzeczy. Zaczynając od rozbudowanej implementacji w jednej klasie, poprzez ich zestaw znajdujący się w jednym pakiecie/module a na sporej części aplikacji kończąc. Ty jako autor diagramu sam decydujesz o tym do jakiego poziomu komponentów chcesz zejść. Istotne jest to, żeby poziom ten był spójny i prezentował wszystkie komponenty na diagramie „z podobnej odległości”.

Przykładowy diagram komponentów

Proszę spojrz na przykładowy diagram komponentów systemu, który może być odpowiedzialny za rezerwację biletów lotniczych:

Możesz na nim zobaczyć kilka komponentów, które są od siebie zależne. Każdy z nich definiuje interfejsy, które pozwalają komunikować się z innymi komponentami. Dla uproszczenia pominąłem opisowe nazwy interfejsów:



Przykładowy diagram komponentów

Diagram wdrożenia

Przedstawiłem Ci już diagram klas i diagram komponentów. Wiesz już, że na system można spojrzeć z różnej odległości zwracając uwagę na coraz mniej szczegółów. Kolejnym stopniem ukrywającym szczegóły może być diagram wdrożenia (ang. *deployment diagram*).

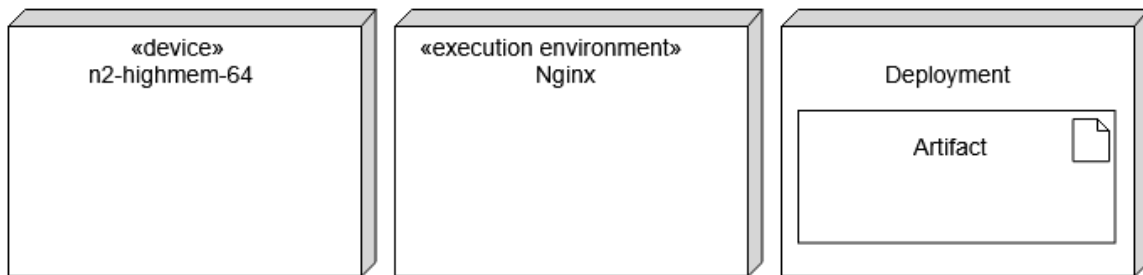
Każdy działający projekt/aplikacja składa się z dwóch niezbędnych elementów. Oprogramowania (ang. *software*) i sprzętu (ang. *hardware*). Zauważ, że żaden z powyżej omówionych diagramów nie poruszał tematyki sprzętu. Tę lukę wypełnia diagram wdrożenia. Diagram wdrożenia służy do odwzorowania zależności pomiędzy oprogramowaniem i/lub sprzętem. To właśnie na diagramie wdrożenia można pokazać sposób w jaki aplikacja/projekt powinien być zainstalowany/wdrożony.

Także tutaj specyfikacja UML pozwala na dużą dowolność jeśli chodzi o szczegóły. Ty jako autor diagramu decydujesz, czy potrzebna jest dokładna specyfikacja poszczególnych elementów sprzętowych, czy zgrubna informacja w zupełności wystarczy.

Na początku swojej przygody z programowaniem ten diagram nie będzie Ci do niczego potrzebny. W późniejszym czasie bardzo pomoże Ci przy rozmowach na temat sposobu wdrożenia projektu.

Elementy diagramu wdrożenia

Przykład poniżej pokazuje elementy, które możesz spotkać na diagramach wdrożenia:



Elementy diagramu wdrożenia

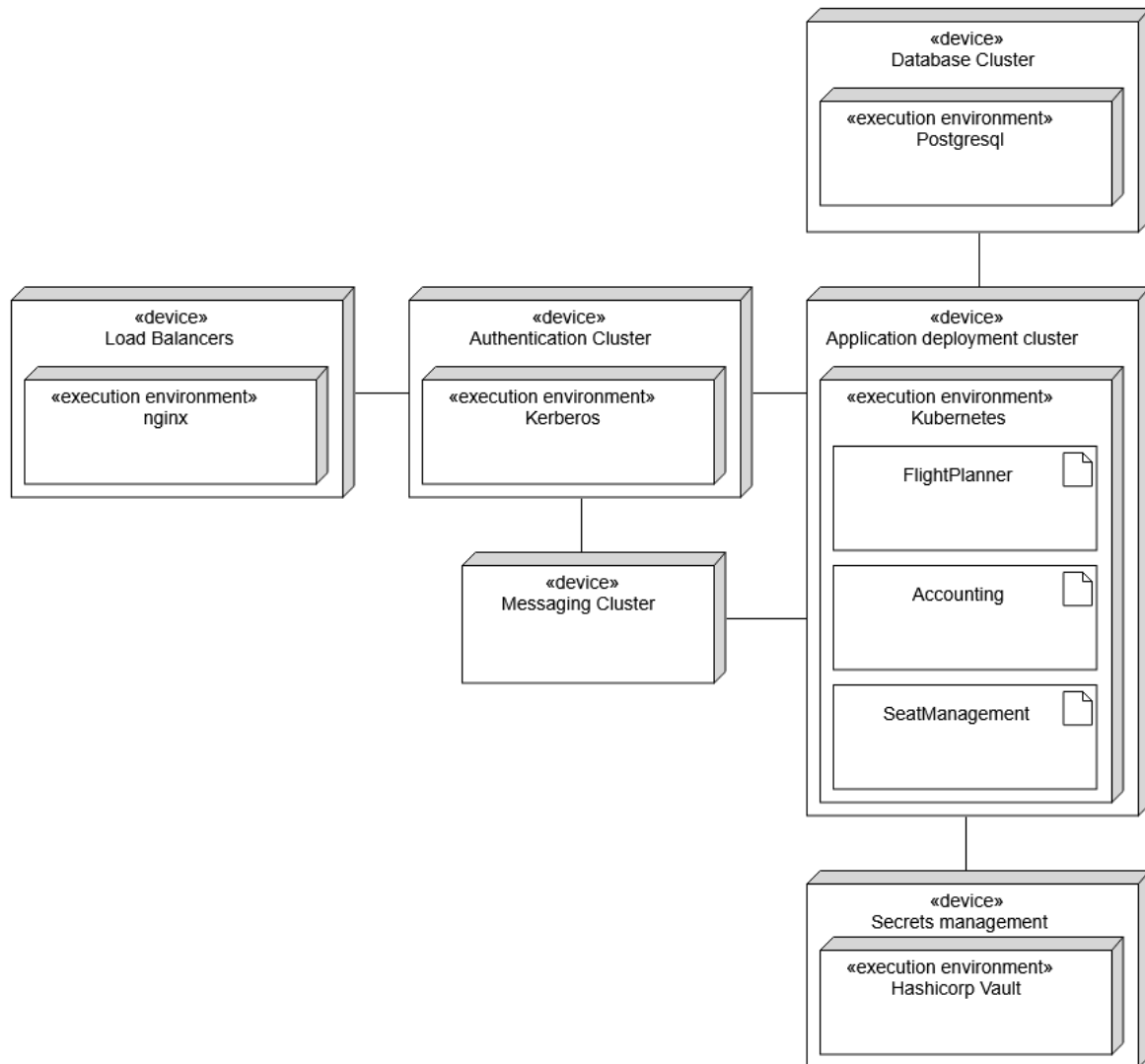
Kolejno od lewej na rysunku możesz zobaczyć:

- serwer typu n2-highmem-64,
- element o nazwie Nginx, który reprezentuje serwer HTTP,
- element Deployment, który wewnątrz zawiera artefakt o nazwie Artifact.

Zauważ, że podobnie jak w przypadku diagramu klas wstępują tu stereotypy, które dodają informacje. Mimo tego, że poszczególne części diagramu reprezentują zupełnie różne rzeczy, UML stosuje jedną graficzną reprezentację. W przypadku tego diagramu zupełnie nie przejmowałbym się sugestiami specyfikacji – w praktyce często spotyka się różnego rodzaju ikonki, które pozwalają lepiej zobrazować poszczególne elementy.

Przykładowy diagram wdrożenia

Proszę spojrz na przykład poniżej, który mógłby być diagramem wdrożenia dla aplikacji pozwalającej na rezerwację biletów:



Przykładowy diagram wdrożenia

Na diagramie wyżej możesz zobaczyć kilka oddzielnych klastrów (zestawów maszyn), przeznaczonych do wdrożenia poszczególnych komponentów. Kreski łączące komponenty obrazują powiązania między nimi.

Diagram sekwencji

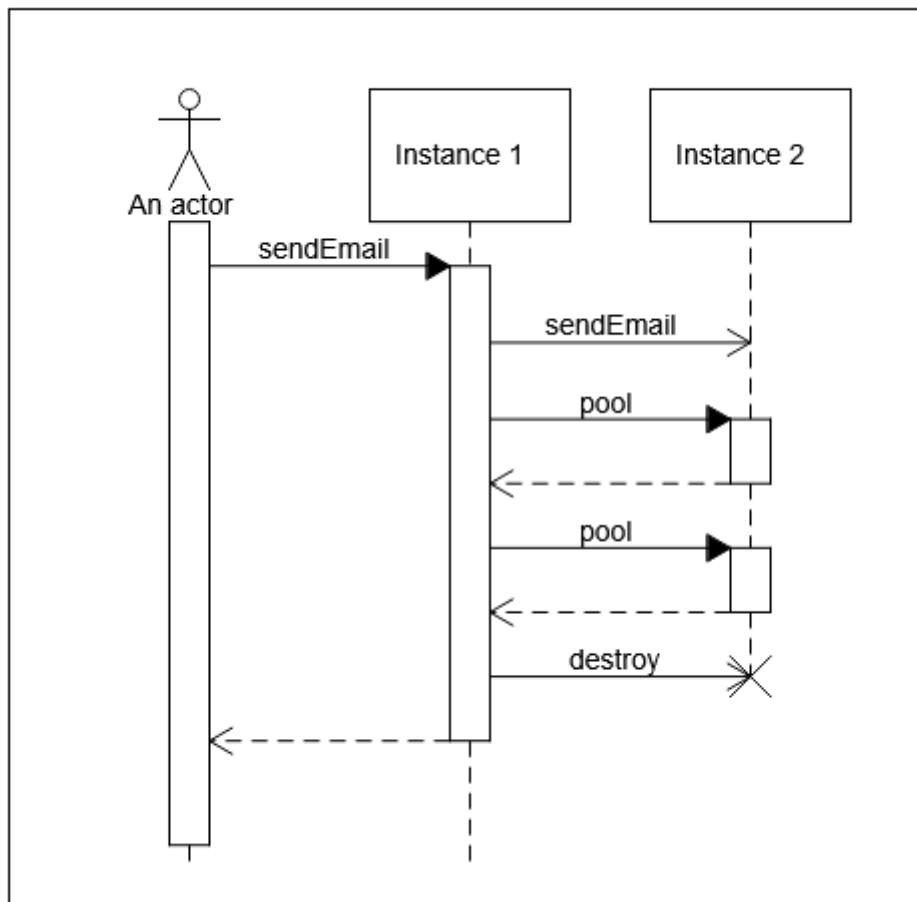
Trzy poprzednie diagramy dotyczyły relacji pomiędzy elementami. Diagram sekwencji (ang. *sequence diagram*) jest jednym z tak zwanych diagramów interakcji. Kładzie on nacisk na komunikację, która odbywa się pomiędzy poszczególnymi klasami/obiektami. Diagram sekwencji pokazuje dokładnie sekwencję wykonania metod w poszczególnych

obiektach. Diagram ten przydaje się do pokazania przebiegu skomplikowanej komunikacji.

Elementy diagramu sekwencji

Każdy z obiektów reprezentowany jest jako prostokąt połączony z pionową kreską. Ta linia oznacza „linię życia” – czas życia obiektu. Na diagramie może występować także tak zwany aktor. Aktor to człowiek albo system, który może brać udział w komunikacji.

Proszę spojrz na przykład:



Przykładowy diagram sekwencji

Wąskie pionowe prostokąty na liniach życia oznaczają czas, w którym dany aktor/obiekt był aktywny. Aktywność była niezbędna do wypełnienia żądania, które dany obiekt wysłał/otrzymał.

Niektóre obiekty mogą żyć krócej niż pozostałe. Koniec życia obiektu zaznaczany jest przez znak X na ich linii życia.

Diagram, który pokazałem powyżej może służyć jako przykład opisujący mechanizm wysyłania wiadomości e-mail. Na początku aktor inicjalizuje proces, Instance 1 obsługuje akcję sendEmail przekazując ją asynchronicznie do Instance 2. Następnie dwukrotnie sprawdza czy wysłanie wiadomości się powiodło, po czym zwraca informację do aktora.

Rodzaje komunikatów

Wiesz już, że pionowe kreski oznaczają linię życia. Im wyżej na diagramie, tym wcześniej coś się wydarzyło. Poziome kreski oznaczają komunikaty. Jak widzisz istnieje kilka rodzajów komunikatów:



Rodzaje komunikatów

Strzałki w lewej kolumnie oznaczają komunikaty synchroniczne. Strzałka z ciągłą linią oznacza wysłanie komunikatu, strzałka z przerywaną linią otrzymanie odpowiedzi. W prawej kolumnie pokazałem strzałkę reprezentującą asynchroniczne wysłanie komunikatu.

Dodatkowe materiały do nauki

Jak wspomniałem na początku artykułu nie było moim zamiarem wyczerpanie tematu. Celowo skupiłem się wyłącznie na diagramach, które moim zdaniem są najczęściej używane. Ponadto pominąłem sporą część możliwości, których nie używałem w praktyce. Właśnie te diagramy były dla mnie najbardziej przydatne w sesjach przy tablicach z kolegami z pracy. Jeśli jednak temat UML Cię zainteresował zapraszam Cię do zapoznania się z zestawem materiałów dodatkowych. Zacznę od materiałów oficjalnych:

- [Oficjalna strona UML'a](#),
- [Specyfikacja UML 2.5.1](#) – jest niezastąpiona jeśli potrzebujesz zajrzeć do źródła i chcesz poznać wszystkie szczegóły, w innym przypadku gorąco nie polecam.

Dodatkowo mam dla Ciebie artykuł podsumowujący [badanie na temat użycia diagramów w praktyce](#).

Uczelnie techniczne często mają osobne kursy poświęcone tematyce UML'a. Czasami jest też tak, że UML zajmuje część wykładu dotyczącego na przykład inżynierii oprogramowania. Przygotowałem dla Ciebie zestaw odnośników do materiałów przygotowanych na uczelniach:

- [Fragment wykładu z UW dotyczący UML'a część I](#),
- [Fragment wykładu z UW dotyczący UML'a część II](#),
- Opis [diagramu klas](#) w materiałach dla studentów AGH,
- Opis [diagramu komponentów](#) w materiałach dla studentów AGH,
- Opis [diagramu wdrożenia](#) w materiałach dla studentów AGH,
- Opis [diagramu sekwencji](#) w materiałach dla studentów AGH,

Na koniec zestawienie linków do artykułów na Wikipedii:

- [Artykuł o UML](#) na polskiej Wikipedii,
- [Artykuł o UML](#) na angielskiej Wikipedii,
- [Diagram klas](#) na angielskiej Wikipedii,
- [Diagram klas](#) na polskiej Wikipedii,
- [Diagram komponentów](#) na angielskiej Wikipedii,
- [Diagram wdrożenia](#) na angielskiej Wikipedii,
- [Diagram sekwencji](#) na angielskiej Wikipedii.

Podsumowanie

Znasz już mój punkt widzenia dotyczący UML'a. Wiesz, że moim zdaniem warto znać podstawy tego języka. Mogą Ci się one przydać w codziennej pracy. Jeśli lubisz pracować w bardziej formalnym środowisku może się okazać, że UML będzie niezastąpiony. Znasz kilka rodzajów diagramów, które mogą być przydatne. Znasz także darmowe narzędzia, które pozwalają na tworzenie diagramów UML.

Mam nadzieję, że artykuł przypadł Ci do gustu. Proszę daj znać w komentarzach co sądzisz o UML'u. Czy Twoim zdaniem znajomość tego języka przydaje się w codziennej pracy? A może to już tylko zaszłość, która powoli odchodzi do lamusa? Jestem ciekawy Twoje opinii.

Dodatkowo, jak zwykle, proszę Cię o podzielenie się odnośnikiem do artykułu ze swoimi znajomymi. W ten sposób pomożesz mi dotrzeć do nowych Czytelników, za co z góry Ci dziękuję. Jeśli nie chcesz pominąć kolejnych artykułów proszę zapisz się do samouczkowego newslettera i polub Samouczka na Facebook'u. To tyle na dzisiaj, trzymaj się i do następnego razu!